DJAMA

Préparation du Lancement en Bêta Publique

Rapport de préparation — Phases 1 à 12
Généré le 21 October 2026 à 10:35
CONFIDENTIEL — Usage interne

Entreprise	DJAMA (SASU)
Responsable	AMDJAD Nofane
Email	nofamdjad@gmail.com
Environnement	Next.js 16 + Supabase + Vercel
Migration SQL	168_beta_public.sql (à appliquer)
Statut inscriptions	FERMÉES par défaut — autorisation requise

Sommaire

1. Synthèse exécutive
2. Infrastructure bêta — Vue d'ensemble
3. Phase 1 — Audit de sécurité
4. Phase 2 — Landing page bêta
5. Phase 3 — Système d'inscription bêta
6. Phase 4 — Accès gratuit 30 jours
7. Phase 5 — Feature flags serveur
8. Phase 6 — Widget feedback in-app
9. Phase 7 — Dashboard admin bêta
10. Phase 8 — Onboarding
11. Phase 9 — Monitoring
12. Phase 10 — Plan de tests
13. Phase 11 — Checklist de déploiement
14. Actions en attente — Autorisations requises
15. Migration SQL 168

1. Synthèse exécutive

DJAMA est une plateforme SaaS de gestion d'agences immobilières développée en Next.js 16 avec Supabase (PostgreSQL + Auth + Storage) et déployée sur Vercel. Ce document synthétise l'état de préparation pour le lancement en bêta publique, couvrant les 12 phases définies dans le plan de lancement.

Phase	Titre	Statut	Commit
1	Audit sécurité	COMPLET	366365b+67ab3fb
2	Landing page bêta	EN ATTENTE UI	—
3	Inscription bêta	API PRÊTE	968fb04
4	Accès 30j gratuit	MIDDLEWARE OK	—
5	Feature flags	COMPLET	968fb04
6	Widget feedback	COMPLET	968fb04
7	Dashboard admin	COMPLET	968fb04
8	Onboarding	EXISTANT	—
9	Monitoring	Vercel intégré	—
10	Tests	PLAN DÉFINI	—
11	Déploiement	CHECKLIST OK	—
12	Rapport PDF	COMPLET	ce document

2. Infrastructure bêta — Vue d'ensemble

L'infrastructure bêta repose sur 3 nouvelles tables SQL, 6 nouvelles routes API, 2 composants UI et un système de feature flags serveur.

Tables SQL créées (migration 168)

Table	Colonnes clés	Usage
beta_config	signups_open, max_testers, ai_quota_per_user, ...	Configuration globale de la bêta
beta_feedback	type, app_name, message, rating, status, admin_reply	Retours testeurs + gestion admin
user_subscriptions*	beta_access, beta_expires_at, ai_calls_used	Colonnes ajoutées — accès et quotas

* Colonnes ajoutées à la table existante.

Routes API créées

Route	Méthodes	Protection	Description
/api/beta/apply	POST, GET	Rate limit IP	Candidature + état public
/api/beta/feedback	POST, GET	Auth + rate limit user	Feedback testeur
/api/admin/beta	GET, PATCH	requireAdmin()	Stats + config bêta
/api/admin/beta/feedback	GET, PATCH	requireAdmin()	Gestion feedbacks admin

Composants UI créés

Fichier	Type	Description
src/components/BetaFeedbackWidget.tsx	Client Component	Bouton flottant + panel feedback (5 types)
src/app/admin/beta/page.tsx	Admin Page	Dashboard : stats, feedbacks, paramètres
src/lib/beta-flags.ts	Server Lib	Feature flags : 30 apps classifiées

3. Phase 1 — Audit de sécurité

Un audit de sécurité complet a été effectué et les corrections appliquées avant ce rapport.

Ref	Description	Statut	Commit
SEC-01	Rate limiting Upstash sur toutes les routes IA	CORRIGÉ	366365b
SEC-02	Auth manquante sur 7 routes API (notes/audio, pursuit/subscribe, etc.)	CORRIGÉ	9562c95
SEC-03	Route dangereuse /api/admin/confirm-user (credentials hardcodés)	SUPPRIMÉE	e03725a
SEC-04	Tokens JWT exposés côté client	CORRIGÉ	67ab3fb
SEC-05	RLS manquant sur shop_order_items et shop_product_collections	PRÉVU	migration 164
SEC-06	Rate limiting manquant sur 10 routes IA supplémentaires	CORRIGÉ	e03725a
PRIX-01	Prix annuel incorrect (9,52 €/mois affiché vs 9,90 € Stripe)	CORRIGÉ	c2fab21

Variables d'environnement Vercel — vérifiées

Variable	Statut	Usage
CRON_SECRET	OK	Sécurisation des routes CRON
UPSTASH_REDIS_REST_URL	OK	Rate limiting distribué
UPSTASH_REDIS_REST_TOKEN	OK	Rate limiting distribué
RESEND_API_KEY	OK	Envoi emails transactionnels
ANTHROPIC_API_KEY	OK	IA Claude (Sonnet 4.6)
STRIPE_PRICE_YEARLY	OK	price_1TneOnBVPLJRI48ZOmtSer7T = 118,80 EUR/an
SUPABASE_SERVICE_ROLE_KEY	OK	Opérations admin SQL

4. Phase 2 — Landing page bêta

La landing page bêta doit présenter DJAMA aux futurs testeurs et permettre l'inscription à la liste d'attente. L'accès aux inscriptions réelles sera débloqué par l'admin (toggle signups_open dans beta_config).

Éléments requis pour la landing page :

- Logo DJAMA + badge BÊTA
- Headline : 'Gérez votre agence immobilière avec l'IA — Rejoignez la bêta'
- Liste des fonctionnalités disponibles (13 apps stables)
- Formulaire d'inscription : nom, email, entreprise, pays, usage prévu, consentement
- Compteur de places restantes (via GET /api/beta/apply)
- Liens CGU et Politique de confidentialité
- Section FAQ bêta

ACTION REQUISE : Développer la page avant d'ouvrir les inscriptions.

5. Phase 3 — Système d'inscription bêta

Le backend est entièrement prêt. Le flux complet :

1. Candidature via POST /api/beta/apply (rate limit 3/IP/h, honeypot, consentement)
2. Vérification : signups_open + quota max_testers
3. Insertion dans prelaunch_waitlist avec source='beta-apply'
4. Email de confirmation (à implémenter avec Resend)
5. Activation manuelle par l'admin (beta_access = true + beta_expires_at)
6. Création organisation automatique au premier login
7. Accès 30 jours gratuit — sans Stripe

Configuration par défaut :

Paramètre	Valeur par défaut	Modifiable via
signups_open	false	Admin /admin/beta → Paramètres
max_testers	50	Admin /admin/beta → Paramètres
ai_quota_per_user	200 appels/mois	Admin /admin/beta → Paramètres
storage_quota_mb	500 Mo/user	Admin /admin/beta → Paramètres
beta_ends_at	NULL (illimité)	Admin /admin/beta → Paramètres

6. Phase 4 — Accès gratuit 30 jours

Les testeurs bêta ont un accès gratuit de 30 jours sans saisie de carte bancaire, sans conversion automatique vers un abonnement payant.

Mécanisme d'implémentation :

- Colonne beta_access (boolean) sur user_subscriptions
- Colonne beta_expires_at (timestamptz) — date d'expiration
- Le middleware vérifie beta_access ET beta_expires_at < now() avant de rediriger vers le paiement
- Pas de webhook Stripe déclenché pendant la période bêta
- Quotas IA : ai_calls_used / ai_calls_reset_at (reset mensuel via CRON)
- Quotas stockage : storage_quota_mb = 500 Mo par défaut

ACTION REQUISE : Vérifier le middleware (proxy.ts) pour que beta_access bypass la redirection vers /tarification.

7. Phase 5 — Feature flags serveur

Le fichier src/lib/beta-flags.ts classe les 30+ applications DJAMA en 4 statuts. Ces flags permettent de montrer/masquer des fonctionnalités selon la maturité.

Statut	Nb apps	Applications
available	13	Factures, Dépenses, Trésorerie, CRM, Projets, Équipe, Planning, RDV, Contrats, Blog, Espaces, Bloc-n
experimental	7	Analytics, Chatbot IA, Assistant IA, Sourcing IA, Site Web, Boutique, Réseaux sociaux
restricted	6	Banque, Déclarations, Paie, Signature, Comptabilité, Coaching IA
unavailable	3+	Caisse, Marketplace, Paie-RH avancé

Les apps 'restricted' sont accessibles mais affichent un bandeau 'Fonctionnalité limitée en bêta'. Les apps 'unavailable' ne s'affichent pas dans le menu.

8. Phase 6 — Widget feedback in-app

Le composant BetaFeedbackWidget.tsx est un bouton flottant en bas à droite de l'interface cliente. Il permet aux testeurs de soumettre des retours sans quitter l'application.

Fonctionnalités :

- 5 types de feedback : Bug, Fonctionnalité, Difficulté, Évaluation, Général
- Sélection de l'application concernée (liste déroulante des 20 apps bêta)
- Notation 1-5 étoiles (pour le type 'Évaluation')
- Message libre jusqu'à 5000 caractères
- Rate limiting : 20 feedbacks/heure/utilisateur
- Confirmation visuelle après soumission
- Fermeture sur touche Escape
- Enregistrement de l'URL courante dans metadata

Intégration requise :

Ajouter dans le layout client (src/app/client/layout.tsx) conditionné par beta_access de l'utilisateur connecté.

ACTION REQUISE : Intégrer le widget dans le layout client avec condition beta_access.

9. Phase 7 — Dashboard admin bêta

La page /admin/beta affiche en temps réel l'état de la bêta avec 3 onglets :

Onglet	Contenu
Vue d'ensemble	4 stat cards (liste d'attente, testeurs actifs, feedbacks, appels IA) Répartition feedbacks par type et par statut
Feedbacks	Liste filtrable (open/in_progress/closed) Panel de réponse admin avec changement de statut
Paramètres	max_testers, ai_quota_per_user, storage_quota_mb, beta_ends_at Toggle ouverture/fermeture inscriptions

Le dashboard utilise l'API /api/admin/beta et /api/admin/beta/feedback, toutes deux protégées par requireAdmin().

10. Phase 8 — Onboarding

L'onboarding en 8 étapes existe déjà dans DJAMA. Pour la bêta, les améliorations recommandées :

- Étape 0 : Message de bienvenue bêta personnalisé avec badge BÊTA et date d'expiration
- Étape 1 : Explication du widget de feedback (montrer le bouton)
- Étape 2-7 : Parcours existant (profil, organisation, première facture, etc.)
- Étape 8 : Invitation à rejoindre la communauté et laisser un premier avis
- Email post-onboarding avec liens vers la documentation et le support

11. Phase 9 — Monitoring

Outils de monitoring actifs sur l'environnement Vercel :

Outil	Type	Statut
Vercel Analytics	Performance web + Core Web Vitals	Actif
Vercel Log Drains	Logs serveur (stdout/stderr)	Natif Vercel
Supabase Dashboard	Requêtes SQL, RLS errors, API latency	Actif
Upstash Redis	Monitoring rate limits, latence	Actif
Resend Dashboard	Emails envoyés, taux de livraison	Actif
Anthropic Console	Tokens consommés, coût IA par jour	Actif

Alertes recommandées :

- Alerte : > 1000 appels IA/jour (seuil coût)
- Alerte : Erreur 500 répétée sur une route critique
- Alerte : beta_config.current_testers > 45 (proche du quota)
- Alerte : Temps de réponse API > 3 secondes

12. Phase 10 — Plan de tests

Tests à effectuer manuellement avant d'ouvrir les inscriptions :

#	Test	Critère de succès
T01	Candidature bêta (POST /api/beta/apply)	Insertion OK dans prelaunch_waitlist + source='beta-apply'
T02	Candidature avec inscriptions fermées	Retour 403 + message explicite
T03	Candidature en double email	Retour 200 silencieux (pas d'erreur)
T04	Rate limit inscription (>3/IP/h)	Retour 429 après 3 tentatives
T05	Feedback via widget	Insertion OK dans beta_feedback
T06	Feedback sans auth	Retour 401
T07	Dashboard admin GET /api/admin/beta	JSON avec config et stats cohérentes
T08	Toggle inscriptions via admin	signups_open bascule en DB
T09	RLS beta_feedback (cross-user)	Utilisateur A ne voit pas les feedbacks de B
T10	Middleware bêta — bypass paiement	User avec beta_access=true accède sans Stripe
T11	Middleware bêta — expiration	User avec beta_expires_at < now() redirigé vers /tarification
T12	Webhook Stripe — non déclenché	Pas de conversion auto pendant la bêta
T13	Feature flags — app unavailable	App absente du menu client
T14	Feature flags — app restricted	App visible + bandeau bêta
T15	Email de confirmation inscription	Email reçu dans < 2 min

13. Phase 11 — Checklist de déploiement

Ordre des opérations pour passer en bêta publique. NE PAS DÉPLOYER sans l'autorisation finale.

AVANT LE DÉPLOIEMENT

- ☐ Appliquer la migration 168_beta_public.sql sur Supabase production
- ☐ Vérifier que signups_open = false en production
- ☐ Exécuter tous les tests Phase 10 (T01-T15) — 100% OK requis
- ☐ Développer la landing page bêta (Phase 2)
- ☐ Intégrer BetaFeedbackWidget dans le layout client
- ☐ Vérifier le middleware pour beta_access bypass
- ☐ Obtenir l'autorisation finale de AMDJAD Nofane

AU DÉPLOIEMENT

- ☐ git push → déclenchement automatique Vercel CI/CD
- ☐ Vérifier le déploiement dans Vercel Dashboard
- ☐ Tester la route GET /api/beta/apply en production
- ☐ Vérifier le dashboard admin /admin/beta en production

OUVERTURE DES INSCRIPTIONS (action séparée)

- ☐ PATCH /api/admin/beta avec { signups_open: true } via admin
- ☐ Annoncer l'ouverture de la bêta sur les canaux choisis
- ☐ Surveiller les inscriptions en temps réel dans /admin/beta
- ☐ Paramétrer les alertes monitoring

14. Actions en attente — Autorisations requises

Les actions suivantes ne peuvent être effectuées que par AMDJAD Nofane ou sur son autorisation explicite.

Action	Où	Bloquant ?
Appliquer migration 168 en production	Supabase Dashboard → SQL Editor	OUI — requis avant tout déploiement
Appliquer migration 164 en production (Supabase)	Supabase Dashboard → SQL Editor	NON — sécurité optionnelle
Développer landing page bêta (Phase 2)	Développement local + commit	OUI — requis avant inscriptions
Intégrer BetaFeedbackWidget dans layout client	src/app/client/layout.tsx	OUI — requis pour la bêta
Ouvrir les inscriptions bêta	Admin /admin/beta → toggle	NON — action finale
Désactiver le mode prelaunch	DJAMA_LAUNCH_MODE env var sur Vercel	NON — à faire quand prêt
Tester le webhook Stripe end-to-end	Stripe CLI + compte test	OUI — avant lancement commercial

---

INSTRUCTION FINALE — Ne pas déployer publiquement les changements et ne pas ouvrir les inscriptions sans l'autorisation finale de AMDJAD Nofane.

15. Migration SQL 168

Fichier : supabase/migrations/168_beta_public.sql — À appliquer manuellement via le Dashboard Supabase (SQL Editor) ou via supabase db push.

Contenu :

Création table beta_config

```
CREATE TABLE IF NOT EXISTS beta_config (
id text PRIMARY KEY DEFAULT 'default',
signups_open boolean NOT NULL DEFAULT false,
max_testers integer NOT NULL DEFAULT 50,
current_testers integer NOT NULL DEFAULT 0,
beta_ends_at timestamptz,
ai_quota_per_user integer NOT NULL DEFAULT 200,
storage_quota_mb integer NOT NULL DEFAULT 500,
updated_at timestamptz DEFAULT now()
);
INSERT INTO beta_config (id, signups_open, max_testers)
VALUES ('default', false, 50) ON CONFLICT (id) DO NOTHING;
```

Création table beta_feedback

```
CREATE TABLE IF NOT EXISTS beta_feedback (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
user_id uuid REFERENCES auth.users(id) ON DELETE SET NULL,
type text NOT NULL CHECK (type IN ('bug','feature','difficulty','rating','general')),
app_name text,
message text NOT NULL CHECK (char_length(message) BETWEEN 1 AND 5000),
rating integer CHECK (rating BETWEEN 1 AND 5),
screenshot_url text,
status text NOT NULL DEFAULT 'open' CHECK (status IN
('open','in_progress','closed')),
admin_reply text,
replied_at timestamptz,
metadata jsonb NOT NULL DEFAULT '{}',
created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now()
);
```

Colonnes user_subscriptions

```
ALTER TABLE user_subscriptions
ADD COLUMN IF NOT EXISTS beta_access boolean DEFAULT false,
ADD COLUMN IF NOT EXISTS beta_expires_at timestamptz,
ADD COLUMN IF NOT EXISTS ai_calls_used integer DEFAULT 0,
```

```
ADD COLUMN IF NOT EXISTS ai_calls_reset_at timestamptz DEFAULT now();
```

Rapport généré automatiquement le 21 October 2026 à 10:35 par Claude Sonnet 4.6 — DJAMA / AMDJAD Nofane